

BILL OF MATERIALS

Component	Description	Quantity
Raspberry Pi 4/5	Single-board computer (SBC) with 8GB RAM	1
Raspberry Pi camera	Raspberry Pi CSI camera module	1
Raspberry Pi CSI cable	CSI ribbon cable for connecting the camera to the Raspberry Pi	1
5V AC-DC power adaptor	5V, 3A AC-DC power adaptor	1

Similarly, the model can be configured to detect a particular person, for example, someone wearing glasses, or a person holding a smartphone. The key advantage of this approach is that the model remains lightweight while providing highly flexible, prompt-based object detection without requiring retraining.

In practical testing, this approach achieves an inference speed of approximately 7.2fps on a MacBook Air, which is impressive for real-time object detection. On a Raspberry Pi 5 (8GB RAM), performance drops slightly to around 5fps, but this remains sufficient for reliable real-time detection in most edge AI applications. Overall, the combination of a lightweight model and prompt-based object identification makes YOLOE an efficient solution for deployment on resource-constrained edge devices. Fig. 1 shows the tested prototype.

The system can be executed on a Raspberry Pi 4 or Raspberry Pi 5 for edge deployment, or it can be run directly on a laptop by installing Python and the required software packages, making development and testing straightforward.

In the prototype presented here, a Raspberry Pi 5 with 8GB RAM was used. The complete list of hardware components required to build the system is provided in Bill of Materials table.

Software and coding

For Python-based development, it is always recommended to work within a virtual environment. A virtual environment isolates system-specific packages from the system-wide Python installation, preventing dependency conflicts and ensuring

that existing applications remain unaffected. Once development is complete, the virtual environment can simply be deactivated, allowing the Python environment to return to its normal configuration.

Create and activate a Python virtual environment.

```
$ python3 -m venv venv
$ source venv/bin/activate
```

Install the required packages, develop and test the application, and, when finished, exit the virtual environment using:

```
$ deactivate
```

The following Python packages represent the minimum software requirements for developing applications with YOLO and YOLOE:

```
pip install ultralytics opencv-python-headless numpy onnx onnxruntime torch torchvision torchaudio
```

The standard installation works well on Raspberry Pi 5 OS. For development and testing, additional packages were installed on a MacBook Air. Depending on the operating system and hardware platform (laptop or desktop), some packages may need to be installed or modified to match the specific environment.

The primary objective of this experiment is to demonstrate that YOLOE simplifies the addition of custom objects for detection without requiring extensive model retraining. Using YOLOE, new objects can

```
from ultralytics import YOLOE
model = YOLOE("yoloe-11s-seg.pt") # Use '...11l' for the large
model or '...11s' for the small model
# Define the object classes to embed into the ONNX model
# names = ["car", "truck", "bus", "motorcycle", "pickup van"]
names = ["person", "glasses", "smartphone", "clock", "banana"]
model.set_classes(names, model.get_text_pe(names))
model.export(format="onnx", imgsz=640)
```

Fig. 2: Process of exporting the customised ONNX model

be introduced into the model and reliably recognised without lengthy training cycles or high-end GPU hardware.

Two approaches are available to achieve this.

1. Object already available in the model. In this approach, the target objects already exist in the pre-trained YOLOE model (for example, yoloe-11s-seg.pt). In such cases, text prompts are sufficient to specialise the model for those objects.

The Text-Prompt-ONNX-Conversion.py script is used to define prompts for objects that are already included in the base model. During the export process, the selected object classes are embedded directly into the ONNX model. As a result, the exported model is lighter, faster, and optimised to detect only the specified objects during inference.

```
Text-Prompt-ONNX-Conversion.py
```

Next, the generated yoloe-11s-seg.onnx model is executed using the YOLOE-Run-Model.py script. The model detects only the embedded object classes, such as person, glasses, smartphone, clock, and banana, making deployment straightforward and efficient, as shown in Fig. 2.

Fig. 3 shows the console output confirming the successful conversion of the YOLOE-11s-seg model to ONNX, optimised with onnxslim.

2. Objects not available in the pretrained model. For objects that are not present in the pre-trained YOLOE model, a visual prompting approach is used. Instead of providing a text prompt, the physical appearance of each object is supplied using two or three images captured from different viewing angles. YOLOE then associates these visual samples with a unique object ID, enabling the object to be recognised later without retraining the model.

This process